CAHIER DES CHARGES — IMMOANALYSE

Projet fil rouge du Module 1 : Python pour l'IA et le Machine Learning

Version : 1.0
Type : Projet pratique de manipulation et de contrôle de la qualité des données
Domaine : Immobilier
Langage principal : Python

1. Contexte du projet

Les entreprises collectent des données provenant de sources et de systèmes différents. Ces données sont souvent incomplètes, dupliquées, mal formatées ou incohérentes.

Dans le secteur immobilier, ces problèmes peuvent concerner les prix, les surfaces, les dates de transaction, les villes et les types de biens. Des données de mauvaise qualité peuvent fausser les analyses et conduire à de mauvaises décisions.

Le projet **ImmoAnalyse** consiste à développer une application Python capable d'importer plusieurs fichiers immobiliers, de contrôler leur qualité, de corriger les anomalies identifiables, d'analyser les données exploitables et de produire des fichiers propres accompagnés d'un rapport de qualité. Le projet utilise trois formats obligatoires : CSV, Excel et JSON. Les données fournies sont volontairement dégradées afin de reproduire différentes difficultés rencontrées lors du traitement de données réelles.

2. Objectif général

Développer un pipeline Python modulaire et reproductible permettant de transformer des données immobilières brutes et hétérogènes en données contrôlées, exploitables et documentées.

3. Objectifs spécifiques

À la fin du projet, chaque participant devra être capable de :

1. Manipuler les structures de données natives de Python.
2. Organiser un programme en fonctions, modules et packages.
3. Comprendre et appliquer les notions essentielles de programmation orientée objet.
4. Lire et écrire des fichiers CSV, Excel et JSON.
5. Manipuler des tableaux numériques avec NumPy.
6. Nettoyer, transformer et analyser des données tabulaires avec Pandas.
7. Définir et implémenter des règles métier de validation.
8. Mesurer la qualité des données à l'aide d'indicateurs chiffrés.
9. Produire des analyses exploratoires dans Jupyter Notebook.
10. Créer un environnement Python reproductible et documenter l'exécution du projet.

4. Périmètre fonctionnel

Le pipeline devra réaliser les opérations suivantes :

1. Charger les fichiers bruts.
2. Identifier la provenance de chaque enregistrement.
3. Uniformiser les noms de colonnes et les formats lorsque cela est possible.
4. Combiner les données provenant des différentes sources.
5. Détecter les doublons et les conflits entre enregistrements.
6. Contrôler les données à l'aide de règles métier explicites.

7. Corriger les anomalies corrigeables sans ambiguïté.
8. Mettre en quarantaine les enregistrements qui ne peuvent pas être exploités de manière fiable.
9. Produire des indicateurs de qualité.
10. Analyser les données retenues.
11. Exporter les données traitées et les rapports.
12. Documenter les résultats et les limites du traitement.

Hors périmètre obligatoire : développement d'une API web, base de données, interface graphique et entraînement d'un modèle de Machine Learning. Ces éléments pourront faire l'objet d'extensions ultérieures.

5. Jeux de données

Le projet s'appuie sur un pack de données repensé pour reproduire la réalité d'un **pipeline multi-sources**. Contrairement à un jeu de données unique, le pack fournit **cinq flux bruts hétérogènes**, chacun représentant un canal de collecte distinct : une plateforme immobilière en ligne, une agence, des annonces de particuliers, des actes notariés et un historique ancien. Chaque flux possède son propre schéma, ses propres conventions de nommage, ses propres formats et son propre niveau de qualité.

L'objectif n'est pas de concaténer ces fichiers, mais de **comprendre leurs différences, les harmoniser et décider ce qui peut être rapproché, corrigé, mis en quarantaine ou écarté**.

5.1 Structure du pack

```
immoanalyse_datasets/
├── sources/
│   ├── plateforme_immo_sn.csv      (CSV, séparateur ';')
│   ├── agence_teranga.xlsx         (Excel, 2 feuilles)
│   ├── particuliers.json           (JSON imbriqué)
│   ├── notaires.csv                (CSV, séparateur ',')
│   └── import_historique.csv       (CSV, séparateur ';', encodage latin-1)
├── referentiels/
│   ├── referentiel_geographique.csv
│   ├── referentiel_categories.csv
│   ├── dictionnaire_donnees.csv
│   └── regles_metier.md
└── README.md
```

5.2 Les cinq sources

Source	Format	Volume	Schéma propre	Qualité
`plateforme_immo_sn.csv`	CSV `;`	~820 lignes	15 colonnes : `ref_plateforme`, `date_mise_en_ligne`, `type_transaction`, `categorie`, `region`, `commune`, `quartier`, `prix`,	Moyenne (champs libres, prix parfois en millions)

			devise, surface, pieces, sdb, description, statut, date_expiration	
agence_teranga.xlsx	Excel (2 feuilles)	~640 lignes	11 colonnes : Code, Date, Region, Ville, Type, Prix_FCFA, Surface_m2, Chambres, SDB, Statut, Commercial	Bonne (peu de champs, pas de géolocalisation)
particuliers.json	JSON imbriqué	~730 objets	Structure {annonce: {id, date, localisation: {...}, details: {...}, statut}}	Faible (beaucoup de manquants, dates françaises)
notaires.csv	CSV ,	~410 lignes	9 colonnes : id_acte, date_acte, region, ville, type_bien, montant_fcfa, superficie, nb_chambres, statut	Très bonne (peu de volume, pas de description)
import_historique.csv	CSV ; latin-1	~530 lignes	9 colonnes : ID, DateVente, Region, Ville, TypeBien, Prix, Surface, Chambres, SallesBain	Mauvaise (encodage, formats anciens, IDs courts)

5.3 Ce qui rend le rapprochement non trivial

- **Aucun schéma n'est identique.** Les noms de colonnes diffèrent (`prix` vs `Prix_FCFA` vs `montant_fcfa` vs `Prix`), les types aussi.
- **Aucun identifiant n'est universel.** Chaque source a son propre identifiant local (`ref_plateforme`, `Code`, `id`, `id_acte`, `ID`). Le rapprochement doit se faire sur des attributs métier (ville + surface + type +

date), pas sur l'identifiant.

- **Les identifiants ne sont pas fiables.** Un même identifiant peut porter des valeurs contradictoires selon la source ; il doit être traité comme un champ à vérifier, pas comme une clé primaire.
- **Les formats sont hétérogènes** : `2024-03-01`, `01/03/2024`, `hier`, `32/13/2024`, `37 965 000 FCFA`, `58.29 millions`, `1,250` (virgule décimale ou séparateur de milliers ?), `quatre-vingt`.
- **Les catégories sont écrites différemment** : `Dakar`, `DAKAR`, `dakar`, `Dkr`, `Guediawaye`, `GUÉDIAWAYE`, `Thies`, `THIÈS`, `Thiès` .
- **Les chevauchements sont partiels.** Certaines annonces apparaissent dans 2 ou 3 sources, d'autres dans une seule. Aucune source ne couvre l'ensemble.

5.4 Ventes et locations

Le pack couvre **ventes** et **locations**. Les locations possèdent une **périodicité de loyer** (mensuelle, trimestrielle, annuelle) et doivent être distinguées des ventes dans tout le pipeline. Un loyer mensuel de 250 000 FCFA n'est pas comparable à un prix de vente de 250 000 FCFA.
Les prix des ventes sont des **prix demandés** (annonces), pas nécessairement des prix de transaction conclue. Cette distinction doit être explicite dans le dictionnaire de données et dans le rapport de qualité.

5.5 Types de biens

Le pack couvre **sept types** : `Appartement`, `Studio`, `Chambre`, `Villa`, `Maison`, `Terrain`, `Local commercial`. Le référentiel `referentiel_categories.csv` fournit les synonymes admis et des fourchettes indicatives de prix et de surface par type.

5.6 Référentiels fournis

- `referentiel_geographique.csv` : 14 régions, ~45 villes, coordonnées de centre.
- `referentiel_categories.csv` : types de biens, synonymes, fourchettes indicatives.
- `dictionnaire_donnees.csv` : description de chaque champ, type attendu, limites, règles associées.
- `regles_metier.md` : règles bloquantes, normalisables, avertissements, avec justification.

5.7 Nature des données

Le pack contient des **données synthétiques** inspirées du contexte immobilier sénégalais. Elles ne représentent **pas** des transactions réelles et ne doivent **pas** servir à estimer les prix du marché. Les coordonnées géographiques sont générées autour des centres-villes avec un bruit contrôlé ; elles ne correspondent à aucune parcelle réelle. Aucun nom réel, numéro de téléphone ou adresse email réel n'est présent.

5.8 Consignes

- **Conserver les fichiers de `sources/` et de `referentiels/` sans modification.** Toutes les opérations sont réalisées sur des copies, dans un dossier `sortie/` distinct.
- **Produire** : données propres, données en quarantaine, rapport de qualité chiffré, exports CSV et JSON (Excel facultatif).
- **Justifier** chaque décision de nettoyage, y compris les décisions de ne pas corriger.

6. Règles métier et contraintes de validation

Les règles ci-dessous constituent une première version du référentiel. Elles devront être validées, documentées et implémentées avant le traitement complet.
Les seuils de plausibilité sont des hypothèses pédagogiques, pas des normes officielles du marché immobilier.

6.1 Contraintes par champ — tableau de remplacement

Champ	Règle métier	Action en cas de violation
identifiant local	Obligatoire dans chaque source	Quarantaine si absent
`type_transaction`	Doit valoir `vente` ou `location` après normalisation	Quarantaine si non interprétable
`prix` (vente)	Numérique strictement positif	Normaliser si possible, sinon quarantaine
`loyer` (location)	Numérique strictement positif	Normaliser si possible, sinon quarantaine
`periodicite`	Valeur de la liste {mensuel, trimestriel, semestriel, annuel}	Signaler ; ne pas deviner
`surface`	Numérique strictement positif	Normaliser si possible, sinon quarantaine
`date_collecte`	Date interprétable, non future par rapport à la date d'exécution	Quarantaine si impossible
`date_mise_a_jour`	Date interprétable, ≥ `date_collecte`	Signaler
`type_bien`	Doit appartenir aux 7 catégories après normalisation	Quarantaine si hors référentiel
`region` / `ville`	Doivent être cohérentes selon `referentiel_geographique.csv`	Signaler ; ne pas corriger sans preuve
`chambres`, `salles_bain`	Entier ≥ 0 ; cohérent avec le type de bien	Signaler
identifiant global (`id_bien`)	Unique après rapprochement	Distinguer doublon exact, doublon probable et conflit

6.2 Seuils de plausibilité — ajouts

Les seuils existants (surface, prix) restent valables. Il faut ajouter :

- **Loyer** : < 25 000 FCFA ou > 5 000 000 FCFA (par mois, seuils à ajuster selon la périodicité) → valeur atypique à vérifier, pas suppression automatique.
- **Prix au m²** : si `prix / surface` sort des fourchettes du référentiel catégoriel → signaler.
- **Nombre de chambres** : > 10 pour un appartement ou un studio → signaler comme atypique.

- **Salles de bain > chambres + 2** → signaler comme atypique.

Ces seuils sont **configurables** (fichier de configuration ou constantes en tête de module), pas codés en dur au milieu du pipeline.

6.3 Normalisation — ajouts

Le pipeline devra également :

- distinguer `vente` et `location` avant toute normalisation de prix ;
- reconnaître et normaliser les **périodicités** (`/mois`, `par mois`, `mensuel`, `mens.` → `mensuel`) ;
- reconnaître les **dates multi-format** (`ISO`, `JJ/MM/AAAA`, `hier`, dates avec mois en lettres) et **ne pas** convertir une date ambiguë sans justification ;
- traiter `"1,250"` comme **ambigu** : ce peut être 1 250 m² (séparateur de milliers) ou 1,25 m² (virgule décimale). La décision doit être documentée par source et par contexte (un terrain de 1,25 m² est invraisemblable ; une villa de 1 250 m² est plausible mais atypique).

7. Mesure de la qualité des données

Chaque règle de validation devra produire des résultats chiffrés.

Les indicateurs obligatoires sont :

- Nombre total d'enregistrements reçus.
- Nombre de valeurs manquantes par colonne.
- Nombre et taux de doublons exacts.
- Nombre d'erreurs de conversion par champ.
- Nombre d'enregistrements conformes et non conformes à chaque règle.
- Nombre d'enregistrements corrigés.
- Nombre d'enregistrements mis en quarantaine.
- Nombre d'enregistrements conservés avec avertissement.
- Nombre d'enregistrements entièrement conformes.

Pour une règle donnée :

Taux de conformité = nombre d'enregistrements conformes / nombre d'enregistrements évaluables × 100.

Le rapport doit préciser le dénominateur utilisé. Les valeurs non évaluables doivent être comptabilisées séparément et ne doivent pas être considérées automatiquement comme conformes. Le taux global de conformité devra reposer sur une définition explicite et éviter de compter plusieurs fois les mêmes enregistrements.

8. Étapes de réalisation

Étape 1 — Python et structures de données

Manipuler des exemples de biens immobiliers avec les variables, conditions, boucles, fonctions, exceptions, listes, dictionnaires, tuples et ensembles.

Livrable : fonctions simples de filtrage, de vérification et de manipulation.

Étape 2 — Programmation modulaire et POO

Créer des modules distincts pour le chargement, la validation et le traitement des données. Utiliser une classe simple pour représenter un bien ou une règle de validation.

Livrable : structure modulaire et responsabilités clairement séparées.

Étape 3 — Chargement des données

Lire les trois formats, harmoniser les noms de colonnes et ajouter une information de provenance pour chaque source.

Livrable : données brutes chargées et inventaire des anomalies initiales.

Étape 4 — Validation et nettoyage

Implémenter les règles métier, les conversions fiables, la détection des doublons et la mise en quarantaine. **Livrable :** données traitées, données en quarantaine et rapport de qualité.

Étape 5 — NumPy et Pandas

Calculer les statistiques descriptives, les prix au mètre carré lorsque les valeurs sont exploitables, les moyennes et médianes ainsi que les résultats par ville et par type de bien.
Livrable : analyses numériques reproductibles.

Étape 6 — Jupyter Notebook

Présenter les distributions, les anomalies, les taux de conformité et les principaux résultats d'analyse.
Livrable : notebook documenté, contenant du code exécutable et des conclusions.

Étape 7 — Exportation et reproductibilité

Exporter les résultats, vérifier les fichiers produits, préparer l'environnement virtuel et documenter l'installation et l'exécution.
Livrable : pipeline exécutable et README complet.

9. Architecture indicative

```
immo-analyse/
├── data/
│   ├── raw/
│   ├── processed/
│   └── quarantine/
├── reports/
│   ├── quality_report.json
│   └── quality_summary.csv
├── notebooks/
│   └── 01_exploration.ipynb
├── src/
│   ├── loaders/
│   ├── validation/
│   ├── processing/
│   ├── analysis/
│   └── main.py
├── tests/
├── requirements.txt
└── README.md
```

Cette architecture est indicative. Il est préférable de commencer simplement, puis d'ajouter des modules au fur et à mesure des besoins.

10. Outils autorisés et recommandés

- Python pour la logique de traitement.
- NumPy pour les calculs numériques.

- Pandas pour la manipulation tabulaire.
- Jupyter Notebook pour l'exploration et la présentation des résultats.
- venv et pip pour l'environnement virtuel et les dépendances.
- Bibliothèques de lecture Excel et JSON adaptées aux formats utilisés.

L'utilisation de scikit-learn, d'une base de données, d'un service cloud ou d'une API externe n'est pas nécessaire pour valider ce module.
Parquet est une extension facultative.

11. Livrables finaux

Chaque équipe devra fournir :

1. Le code source organisé.
2. Les règles métier documentées et implémentées.
3. Les fichiers de données traitées.
4. Les fichiers mis en quarantaine avec les motifs associés.
5. Un rapport de qualité chiffré.
6. Un notebook d'analyse exploratoire.
7. Les fichiers exportés en CSV et JSON ; Excel pourra également être produit.
8. Un environnement reproductible et un fichier de dépendances.
9. Un README expliquant l'installation, l'exécution, les règles et les résultats.
10. Une démonstration du pipeline complet.

12. Critères de validation du projet

Le projet sera considéré comme validé si :

- les trois formats obligatoires sont correctement lus ;
- les règles métier sont explicites et testables ;
- les anomalies attendues sont détectées ;
- les corrections réalisées sont justifiées ;
- les doublons et les conflits sont traités selon une stratégie documentée ;
- les indicateurs de qualité sont calculés correctement ;
- les données brutes restent préservées ;
- les exports sont lisibles et cohérents ;
- le notebook est reproductible ;
- un autre membre peut installer et exécuter le projet à partir du README ;
- les participants savent expliquer et défendre leurs choix techniques.

Les tests devront inclure des cas conformes, des cas invalides et des cas limites. La validation ne reposera pas uniquement sur l'absence d'erreurs à l'exécution.

13. Principe de travail

Le projet doit être réalisé progressivement, en reliant chaque nouvelle notion Python à une fonctionnalité concrète.
Il ne s'agit pas simplement de nettoyer un fichier ou d'obtenir un résultat final. Il faut démontrer que l'on sait écrire du code compréhensible, appliquer des règles explicites, mesurer la qualité des données et reproduire les résultats.
Résultat attendu : une application Python capable de transformer des données immobilières hétérogènes en données contrôlées, accompagnées d'indicateurs de qualité fiables et d'analyses documentées.